MEMORIA PROYECTO CLASIFICADOR DE POSICIONES DE MANO



APRENDIZAJE AUTOMÁTICO Y BIG DATA
CURSO 2025-2026

AUTOR

CARLOS RODRÍGUEZ ROMERA

FACULTAD DE INFORMÁTICA - UNIVERSIDAD COMPLUTENSE DE MADRID

1. Introducción y descripción del problema **3**
2. Objetivos del proyecto **4**
3. Descripción y análisis del conjunto de datos **5**
4. Preprocesado y limpieza de datos **6**
5. Técnicas y modelos aplicados **9**
6. Metodología Experimental **10**
7. Métricas utilizadas para la evaluación **11**
8. Resultados obtenidos **12**
9. Comparativa y análisis crítico de resultados **15**
10. Limitaciones del trabajo y posibles mejoras futuras **16**

1. Introducción y descripción del problema

Para el desarrollo de este trabajo se ha seleccionado un conjunto de datos que, además de resultar motivador, plantea un serio reto técnico. Se ha utilizado un dataset que puede ser encontrado en Kaggle, basado en imágenes de las posiciones de manos (o sellos) de la serie de animación Naruto: [Naruto Hand Sign Dataset].

A modo de contexto, en esta serie de animación los personajes deben hacer combinaciones de estos símbolos para poder invocar ataques o habilidades. Lo que, a pesar de su descontinuación, quedó como uno de los símbolos de la serie. El problema de clasificación consistirá en clasificar estos 12 símbolos, que se pueden ver en la imagen 1, añadiendo un elemento de control llamado zero.

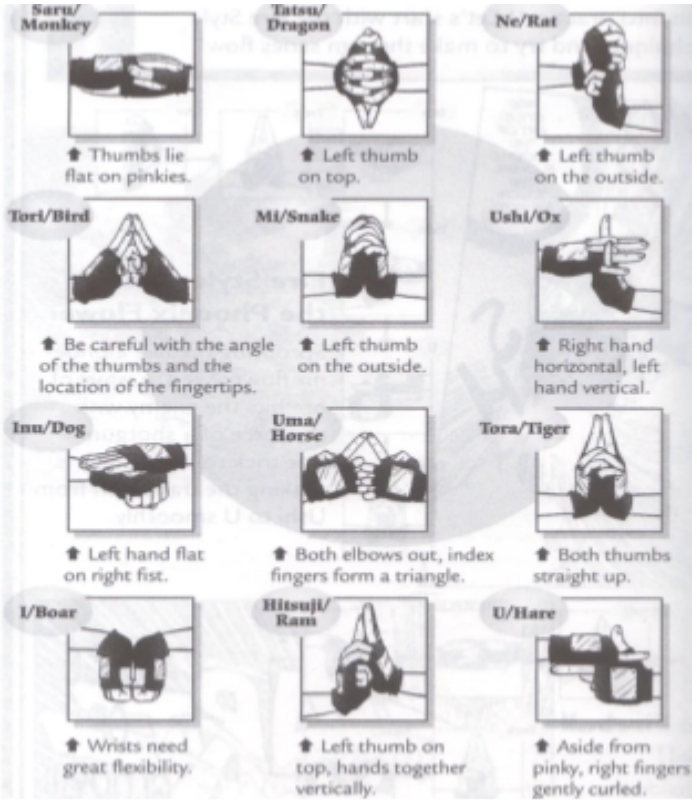


Imagen 1

Esto podría conducir a varios problemas técnicos, ya que la clasificación de imágenes en bruto requiere del uso de redes neuronales convolucionales, las cuales distan del objetivo de la práctica. Por suerte, recientemente Google lanzó un proyecto open source llamado [media pipe]. Este proyecto consiste en un sistema capaz de transformar imágenes en un vector de coordenadas. La forma del vector y la manera en la que esto se realiza puede variar en función de lo que se desee trackear, para el proyecto nos interesa trackear la [posición de las manos]. Más adelante, en el apartado sobre el preprocesado de datos se hablará de cómo se realiza esta tarea.

2. Objetivos del proyecto

Este proyecto consta de varios objetivos diferentes, basados en la creación y el entendimiento de esta clase de sistemas. El primero de los objetivos es entender el funcionamiento y utilidad de un sistema como media pipe. Es un sistema realmente potente y entender cómo funciona por dentro será de gran utilidad tanto en este como en muchos otros proyectos.

El segundo de los objetivos es crear un perceptrón multicapa, o MLP, utilizando la biblioteca NumPy, conocer cómo esta funciona y cómo se puede optimizar. Este es el objetivo principal y entorno al cual se establece el desarrollo completo de la práctica, siendo también el objetivo más complejo.

El último de los objetivos de la práctica es optimizar el rendimiento del modelo, buscando maximizar su capacidad de generalización. Esto puede dar pie a la creación de un traductor de lenguaje de signos en tiempo real utilizando media pipe y numpy, pero por el tamaño necesario del conjunto de datos, y la complejidad del problema, esto era inasumible para esta práctica.

3. Descripción y análisis del conjunto de datos

El dataset seleccionado cuenta con 2245 imágenes divididas en dos grupos, train y test. Test consta de 86 imágenes, mientras que en train hay 2159. Ambos subconjuntos están divididos en 13 clases diferentes, las 12 que se pueden ver en la imagen 1, y zero, una clase número 13 de control, que consiste en chocar los dos puños de frente.

Las clases se encuentran bastante desbalanceadas dentro del dataset. En el conjunto de entrenamiento hay el doble de muestras de la clase *dog* que de algunas otras clases. El conjunto de test también se encuentra bastante desbalanceado, ya que hay clases con hasta 10 imágenes mientras que otras solo cuentan con 5.

Además del desbalanceo se puede observar que las imágenes de una misma clase, en el conjunto de train, están extraídas de uno o varios videos similares, por lo que la posición absoluta de las manos será similar en todas ellas. Esto causará grandes problemas cuando se intente validar un modelo, ya que se trata de un data leakage por correlación temporal, y causará un accuracy muy alto en la validación que no se verá reflejado en los test. Sobre este problema, y las técnicas utilizadas para mitigarlo se hablará más adelante.

4. Preprocesado y limpieza de datos

Realmente gran parte del trabajo de preprocesado de datos de este ejercicio recae sobre media pipe, que se encarga de rastrear las manos, y devuelve una tupla de 21 puntos por mano, cada uno de los puntos con sus coordenadas absolutas X, Y y Z en una escala de 0 a 1. Media pipe es realmente muy configurable, a pesar de que no todos los parámetros son útiles para el trabajo que queremos realizar. Los parámetros más interesantes para este trabajo son el del número de manos a detectar, que lo estableceremos en 2, y el min_detection_confidence. Este último es un parámetro que fija el umbral mínimo de confianza para poder afirmar que se ha encontrado una mano. Si es muy bajo el sistema tenderá a ver manos donde no las hay, en pliegues de camisetas, manchas en las paredes… Si es muy alto habrá fotos con manos claramente diferenciables en las que el sistema dirá que no ha podido encontrar manos. Tras hacer varias pruebas el rango óptimo está entre 0.5 y 0.75, por lo que usaremos 0.6.

Por dentro media pipe utiliza dos redes neuronales que trabajan conjuntamente, en caso de hablar del módulo de seguimiento de manos. La primera de las redes detecta dónde pueden encontrarse las manos en la imagen, o lo que cree que podría ser una mano, con un umbral de confianza. Como se ha comentado anteriormente, en este trabajo se ha usado 0.6 para este valor, todo lo que quede por debajo se descarta automáticamente. Luego estos datos se pasan a la segunda red neuronal, que es la encargada de extraer los puntos más “importantes” dentro de las manos.

Se ha adjuntado un pequeño problema que dadas dos imágenes que extraje a mano del dataset, las pasa por media pipe y superpone los resultados a las mismas. Está adjuntado en la sección de código, nombrado como “mediapipe.py”. Los resultados pueden verse en las imágenes 2 y 3, que son la salida de este proceso.



Imagen 2

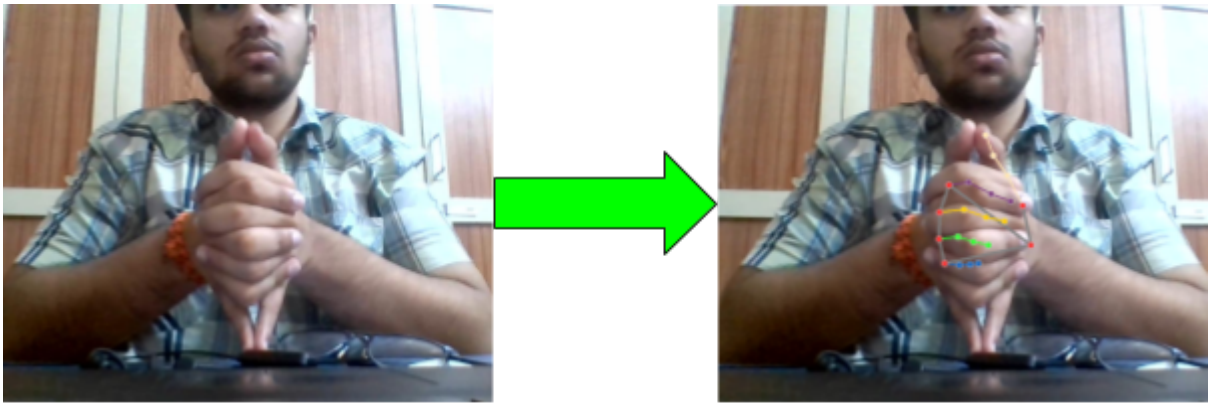


Imagen 3

Como puede observarse media pipe no termina de detectar correctamente los puntos en las manos, de hecho sólo detecta una mano en lugar de las dos configuradas inicialmente. Si nos fijamos en detalle en la imagen 3 se puede ver cómo, salvo el pulgar, ningún dedo está siendo reconocido como debería. Esto puede parecer un problema muy grande, ya que puede introducir muchísimo ruido, pero tras investigar un poco se ha llegado a la conclusión de que se trata de un error sistemático. Es decir, en las imágenes de dragon, como la imagen 3, en la mayoría de los casos la representación con media pipe será mala, pero siguiendo un mismo patrón. Como se verá más adelante esto puede generar ciertos problemas cuando hay clases cuya representación cae en el mismo tipo de errores sistemáticos, haciendo que sea complicado diferenciarlas.

Con esta idea en mente, y con la intención de no tener que pasar por media pipe cada muestra cada vez que se quiera ejecutar el algoritmo se ha creado un script que transforma los datos de entrada a un csv automáticamente TODO PONER NOMBRE. Cada fila de este csv, es decir, cada imágen, consta de 126 columnas (21 x 2 manos x 3 coordenadas) y una variable objetivo. Como se ha visto previamente, el sistema no siempre es capaz de detectar manos en las imágenes, por lo que tras aplicar este script se quedan 1878 imágenes en train y 64 en test, ya que se han eliminado las imágenes en las que no se ha podido detectar nada.

Técnicamente los datos de salida de media pipe ya están normalizados, por lo que no será necesario escalarlos ni nada similar. En cambio sí que se ha tenido que aplicar one-hot encoding para poder obtener la variable objetivo en un array de 0s y 1s. Para ello se ha utilizado LabelBinarizer de Scikit-Learn.

Además de esto, se ha aplicado un Hold-out validation, por lo que, antes de iniciar la red neuronal se aparta un porcentaje de datos con train_test_split. Se han seleccionado un 15% de los datos para este propósito. Debido a que las clases del problema están desbalanceadas se ha realizado una estratificación en función de la y, con el fin de evitar tener conjuntos de validación con muy pocos, o demasiados ejemplos de una clase. Esto se ha realizado ya que en esta clase de problemas no procede hacer un cross validation debido al enorme número de iteraciones de las que constan.

Tras completar el problema, y con el fin de mejorar la precisión de test, que estaba estancada en torno a un 70% se decidió reescalar todos los datos. Esto se hizo en un pequeño script, y todas las entradas se pasaron a posición relativa con respecto a la muñeca. El fin de este proceso es evitar el gran impacto que estaba teniendo en el sistema el data leakage temporal. El data leakage temporal se sufre ya que las imágenes habían sido extraídas de un mismo video en train, por lo que la mayoría de las mismas tienen coordenadas similares en sus puntos. Como media pipe extrae estas coordenadas en valor absoluto, con respecto a su posición en la imagen, el sistema estaba aprendiendo más por la posición con respecto a la imágen que con respecto a la propia colocación de las manos. Por ejemplo (no es real), si todas las imágenes del símbolo del dragón están tomadas en la mitad derecha, todos sus X serán cercanos a 1, esto lo aprende el sistema, y si en test se evalúa un dragón tomado a la izquierda, como tendrá un X mucho menor, el sistema lo clasifica en otras clases con X más a la izquierda. Esto ayudará en gran medida a que el sistema valore la posición de los dedos con respecto a la mano, y no con respecto a su posición en pantalla.

5. Técnicas y modelos aplicados

Para la resolución del ejercicio se ha decidido utilizar un perceptrón multicapa o MLP de 3 capas, como los trabajados en clase. Consta de una capa de entrada con 126 neuronas, una por cada uno de los valores X, Y o Z de cada uno de los 42 puntos. Tras esta hay una capa oculta, con un número de neuronas que fue optimizado, pudiendo tener valores entre 25 y 80 neuronas. En el caso con mejor accuracy en validación, esta capa tenía 25 neuronas. Por último hay una capa de salida con 13 neuronas.

Se ha implementado un algoritmo de forward propagation, muy similar al ya visto en clase y utilizado en alguna práctica. Se calcula la activación de una neurona en función de la combinación lineal de los pesos y las entradas. Para acotar las salidas se emplea la función sigmoide, extraída directamente de las prácticas realizadas a lo largo del curso, cuya definición matemática es la siguiente:

$$g(z) = \frac{1}{1 + e^{-z}}$$

De cara a medir el error cometido por el sistema se ha utilizado una función de coste multivariable similar a la vista en clase. Además se ha utilizado un factor regulador λ. En versiones prematuras del algoritmo este valor tendía a valores extremadamente cercanos a 0, es decir, no regularizar, por lo que había un gran sobreajuste a los ejemplos de entrenamiento (y a los de validación por la correlación temporal). Función de coste:

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K}\left[y_k^{(i)}\log((h_\Theta(x^{(i)}))_k) + (1-y_k^{(i)})\log(1-(h_\Theta(x^{(i)}))_k)\right] + \frac{\lambda}{2m}\sum_{l=1}^{L-1}\sum_{i=1}^{s_l}\sum_{j=1}^{s_{l+1}}(\Theta_{j,i}^{(l)})^2$$

Por último, la actualización de las matrices de pesos se realiza mediante el algoritmo de backpropagation, calculando analíticamente las derivadas parciales de la función de coste respecto a los pesos. Estos gradientes alimentan a un algoritmo de optimización avanzado de la librería SciPy (TNC), el cual itera hasta 250 veces para encontrar el mínimo global, garantizando así la máxima precisión del modelo final. Por esto, la ejecución de todo el programa puede ser algo lenta.

6. Metodología Experimental

De cara a seleccionar el mejor modelo se ha seguido el principio Hold-Out validation. Esto se debe a que con 250 iteraciones, realizar un cross validation sería inasumible en coste computacional, ya que si se seleccionarán 10 particiones (folds) por ejemplo, para un solo modelo habría 2500 iteraciones totales, lo cual supondría 75000 iteraciones al valorar todos los modelos. Para seguir este principio, y como se ha comentado en el apartado de preprocesado de datos, se separan un 15% de los datos de test para hacer validación.

Con el fin de seleccionar el mejor modelo se buscó optimizar los hiperparametros del número de neuronas en la capa oculta y el factor de regularización en la función de coste. En este sentido se han probado los siguientes valores, para el número de neuronas se ha probado con 25, 35, 50, 66 y 80 neuronas; mientras que para el factor de regulación se ha probado con 0.01, 0.03, 0.1, 0.3, 1.0 y 3.0.

En versiones tempranas, cuando todavía no se habían puesto los datos de manera relativa, el modelo seleccionaba como mejores hiperparametros 35 neuronas y 0.01 de factor de regulación. Con esta configuración se obtenía un 70% de accuracy en test. En versiones posteriores, después de pasar las coordenadas a modo relativo los mejores hiperparametros fueron 25 neuronas y 0.01 de factor de regulación, empatado con 35 neuronas y 0.03 de factor de regulación con un 96.81% de accuracy en la validación. Esta versión tenía un accuracy cercano al 85% en test. Tras estratificar a la hora de extraer el conjunto de validación se obtuvo que los mejores hiperparametros eran 35 neuronas nuevamente y un factor de regulación de 0.1.

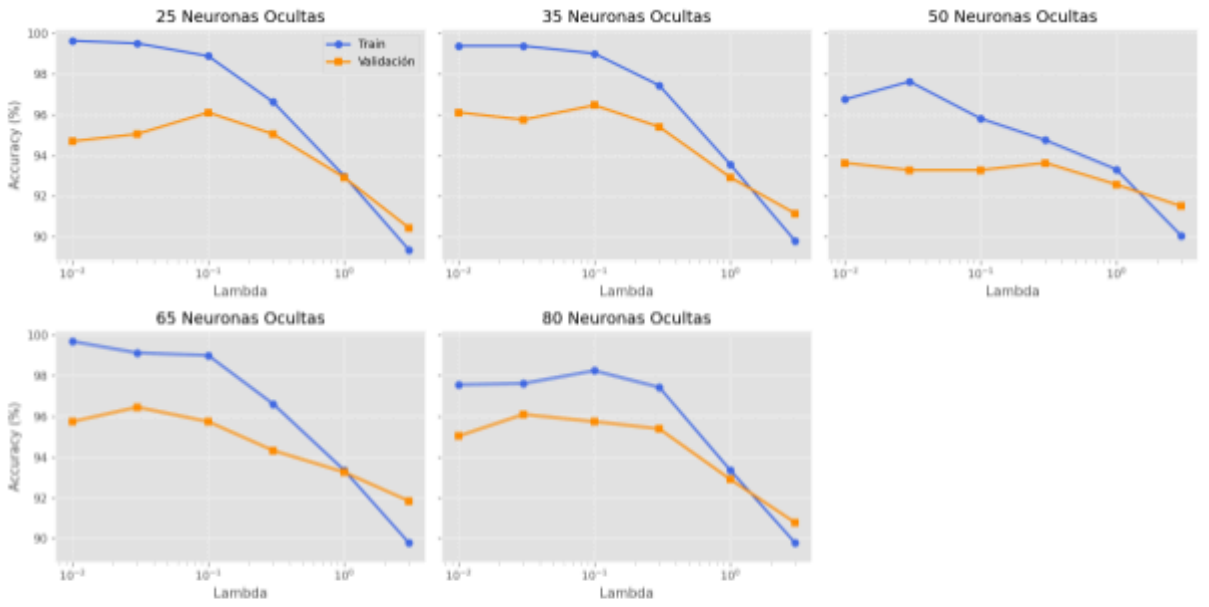
7. Métricas utilizadas para la evaluación

De cara a evaluar el modelo se han utilizado diferentes métricas. A la hora de obtener el mejor modelo de entre todos los candidatos se ha utilizado el Accuracy. Con esta métrica se han construido las diferentes curvas de validación del modelo. Debido a la cantidad de configuraciones diferentes se usan muestran varias curvas de validación, en concreto, para cada número de neuronas de la capa oculta hay una curva en función del factor de regulación.

Para el mejor modelo se han usado varias métricas diferentes, de cara a valorar correctamente su rendimiento. En primer lugar se usa su f1-score, debido al gran desbalanceo que sufren las clases de test después de eliminar las imágenes en las que no se podían detectar manos. Además de esto, se ha obtenido la matriz de confusión, con la que se pueden ver claramente dónde comete más errores el sistema y las clases que mejor predice.

8. Resultados obtenidos

En el proceso de seleccionar el mejor modelo se han obtenido las siguiente gráficas de validación:



Gráfica 1

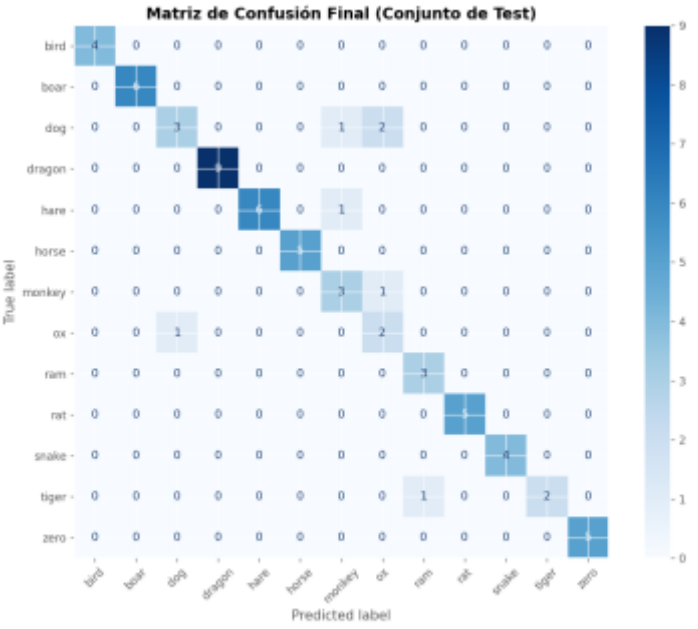
Como se puede observar en la imágen, uno de los grandes logros obtenidos es que el sistema no baja del 90% de accuracy en la valoración, lo cual son valores realmente altos para tener 13 clases. En la imágen no se puede apreciar correctamente cuál es el mejor modelo, ya que la diferencia entre valores es del órden de décimas, pero sí que se puede ver que varios modelos están muy cerca de ser el mejor.

Finalmente el mejor modelo obtenido es aquel con 35 neuronas en la capa oculta y un factor de regulación de 0.1. Esto nos indica que el sistema apuesta por la simpleza frente a modelos con muchas neuronas en la capa intermedia, y por muy poca regulación. Tras reentrenar el modelo con el conjunto entero de train se ha obtenido el siguiente resultado:

	precision	recal	f1-score	support
bird	*1.00*	*1.00*	*1.00*	*4*
boar	*1.00*	*1.00*	*1.00*	*6*
dog	*0.75*	*0.50*	*0.60*	*6*
dragon	*1.00*	*1.00*	*1.00*	*9*
hare	*1.00*	*0.86*	*0.92*	*7*
horse	*1.00*	*1.00*	*1.00*	*5*
monkey	*0.60*	*0.75*	*0.67*	*4*
ox	*0.40*	*0.67*	*0.50*	*3*
ram	*0.75*	*1.00*	*0.86*	*3*
rat	*1.00*	*1.00*	*1.00*	*5*
snake	*1.00*	*1.00*	*1.00*	*4*
tiger	*1.00*	*0.67*	*0.80*	*3*
zero	*1.00*	*1.00*	*1.00*	*5*
accuracy			*0.89*	*64*
marco avg	*0.88*	*0.88*	*0.87*	*64*
weighted avg	*0.91*	*0.89*	*0.89*	*64*

Como se puede observar en la tabla, a pesar de tener solo 64 entradas de test el sistema es capaz de alcanzar un 88% de accuracy y un f1-score de 0.87. Esta es la matriz de confusión que se ha obtenido para esos resultados:

Como se puede observar en la matriz de confusión, en términos generales el sistema tiende a predecir bien, quitando algunos casos. Los mayores errores se producen entre la clase perro y la clase buey. Además hay algún que otro error concreto, con las clases carnero y tigre, liebre, perro y mono.



Gráfica 2

9. Comparativa y análisis crítico de resultados

Como se puede ver en el apartado anterior, los resultados obtenidos por el sistema son excepcionales. Si lo comparamos con un predictor aleatorio por ejemplo, este obtendría un 7 o 8% de accuracy, mientras que la arquitectura propuesta llega hasta el 88% lo cual es un salto increíble. Otros posibles sistemas, como knn, no serían capaces de lidiar correctamente con el error sistemático que posee este sistema.

Además de eso, si nos fijamos en las imágenes de las clases que más confunde, que son perro con buey, las posiciones de las muñecas, y de las manos se parece ciertamente, lo que puede que nos esté conduciendo a error.

Como última reflexión, se puede apreciar cómo el sistema sigue sufriendo por el data leakage por correlación temporal a pesar de haber pasado los datos a valores relativos. Esto deja un mensaje claro, y es que es mucho más importante tener unos datos sólidos y lo mejor estructurados posible, antes que un sistema perfecto. En el último apartado se propone una posible solución a esto, pero es inabordable.

10. Limitaciones del trabajo y posibles mejoras futuras

La mayor limitación del trabajo es la vista al inicio, que se debe al error sistemático que comete media pipe con las imágenes. Esto ha lastrado en gran medida el desarrollo del proyecto, a pesar de que los resultados son bastante buenos. Como posible mejora en este campo se podría rehacer el dataset por completo, con imágenes más claras, con mejor iluminación y una separación un poco mayor entre los dedos, pero quizá esto no solucione nada. En cualquier caso conlleva un esfuerzo tan grande que se puede desestimar.

Como ampliación futura se puede integrar OpenCv, pero excedería el alcance temporal de este proyecto. Media pipe tiene soporte para ser usado con imágenes en tiempo real, por lo que la integración no sería excesivamente compleja.

Como apunte final, todo lo realizado en este proyecto es escalable para poder llegar a lo que supondría el límite máximo de este tipo de arquitectura, que sería un traductor de lenguaje de signos en tiempo real. Pero el tamaño del hipotético dataset es inasumible para esta clase de proyectos.